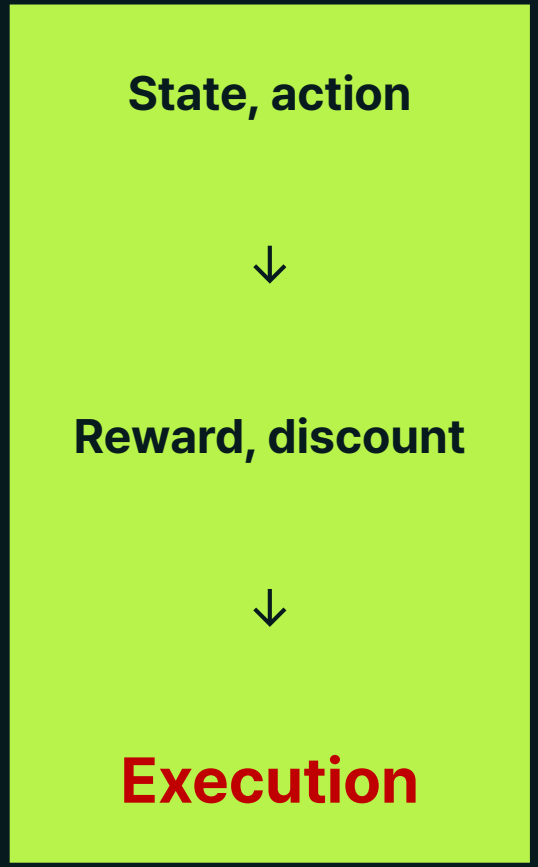
Reinforcement Learning From Forecast to Action

From ‘what will happen’ to ‘what to do now’



Financial ML · Week 13

This week runs in four steps

From basics to core concepts, cases, then summary

01	Basics	**Three learning types** Supervised, unsupervised, RL; updates
02	Core concepts	**State to policy** Discounting, Q-learning, execution
03	Cases	**Games vs. markets** AlphaGo, Knight Capital, Almgren–Chriss
04	Summary	**Reward, validation** Check Q&A

Accurate forecasts still need the cost of action weighed

01

Basics

How RL differs from other learning methods

RL learns from rewards, not answers

Three ways machine learning learns

SUPERVISED

Supervised

- Learns from labeled answers.
- e.g. next-day return

UNSUPERVISED

Unsupervised

- Finds structure, no answers.
- e.g. grouping stocks

REINFORCEMENT

Reinforcement Learning

- Learns from rewards for actions.
- e.g. order slicing

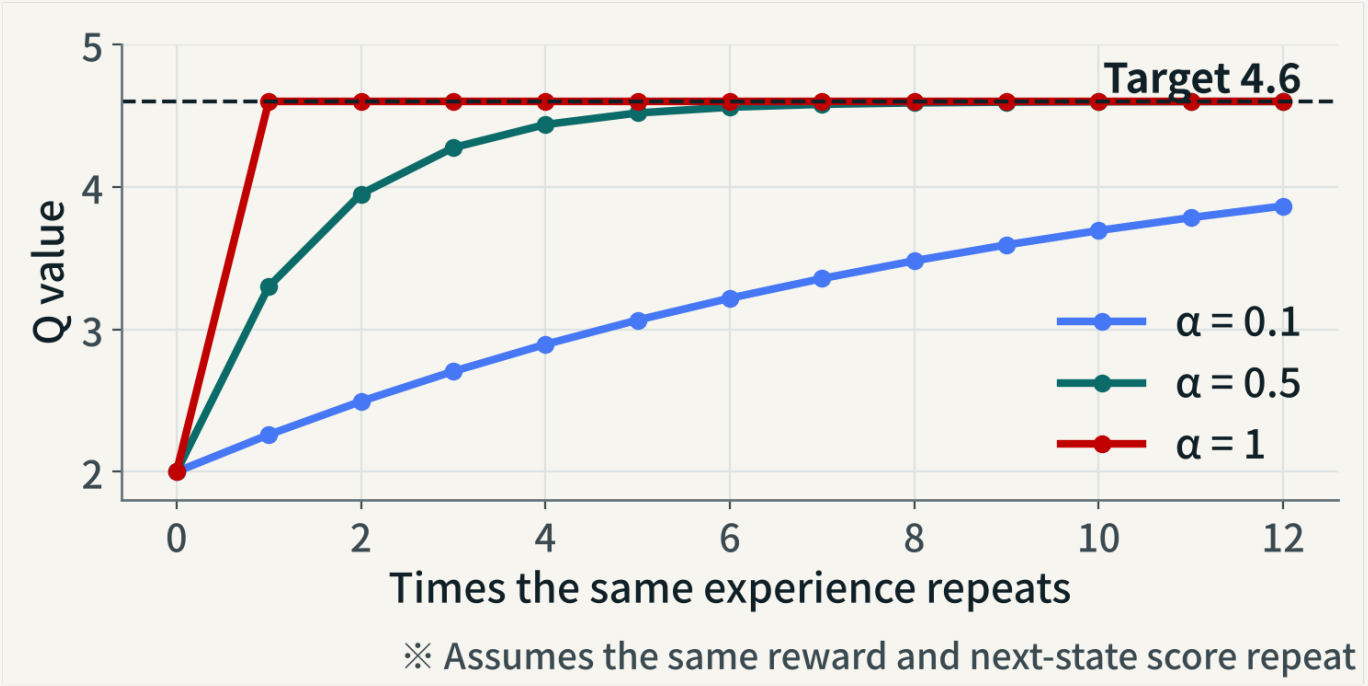
In RL, my action changes the data I see next

Repeating an experience moves Q toward the target

The main deck's Q-learning example (target 4.6), repeated

Learning rate α

- At α=1, it jumps to the target.
- A small α moves slowly, with less noise.



α trades speed for stability

02

State · Action · Reward

Source concepts: discounting, exploration, Q-learning, execution

RL chooses actions instead of guessing answers

Like a game: pick paths, get scores, and revise strategy from outcomes

SUPERVISED

Supervised

- Asks what tomorrow's price will be.
- Learns from prediction error.

What will happen?

REINFORCEMENT

Reinforcement Learning

- Asks how much to buy now and what to do next.
- Revises policy from outcomes.

What to do?

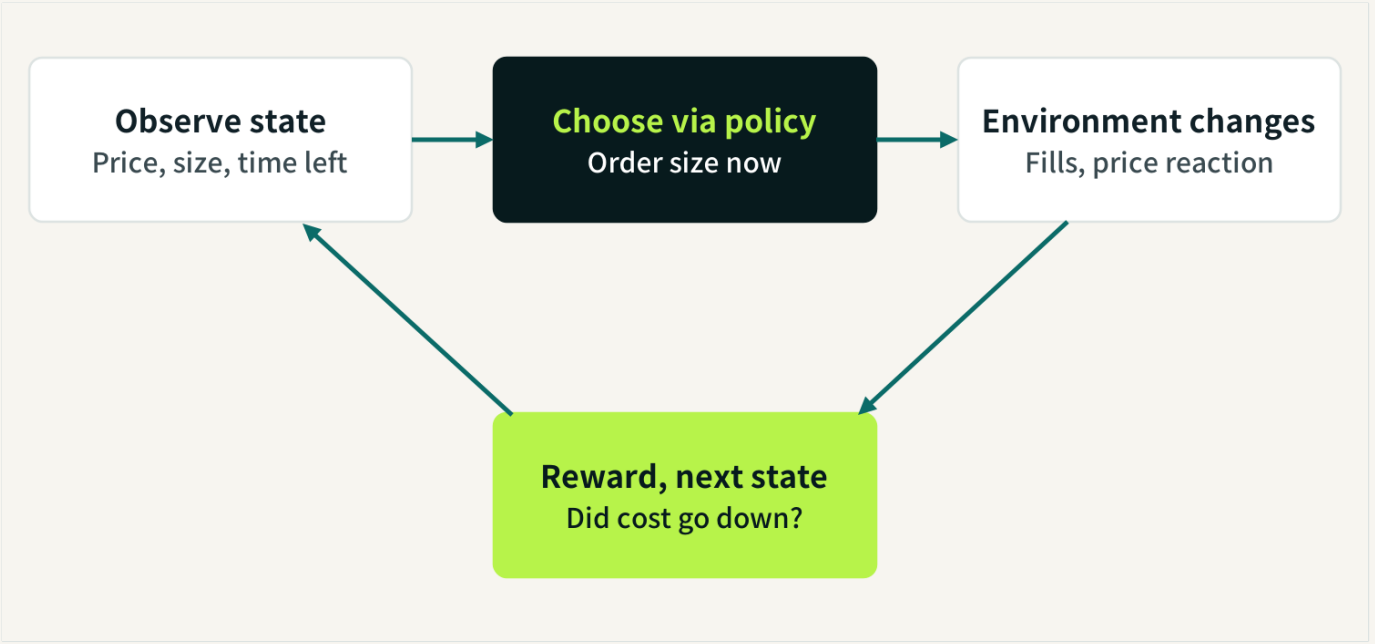
Markets cannot be re-run, so we use simulators; a wrong simulator means a failed policy

Current actions change the next state, in a loop

Example: a program splitting a large order

Key

- A big order can move the very price you want to buy at.
- So multi-step outcomes are judged together.



Unlike prediction, actions change the world

Six terms defined with one execution case

Map each term to splitting a large order

Term	Plain meaning	Execution example
Agent	**Who picks actions**	**Order-splitting program**
State	**What is known now**	**Shares, time left, quotes**
Action	**What to do now**	**Order 100 or 500 shares**
Reward	**Score of last choice**	**Negative exec. cost**
Policy	**Situation → action rule**	**Order more when liquid**
Environment	**World reacting to actions**	**Market or simulator**

The best action depends on the discount

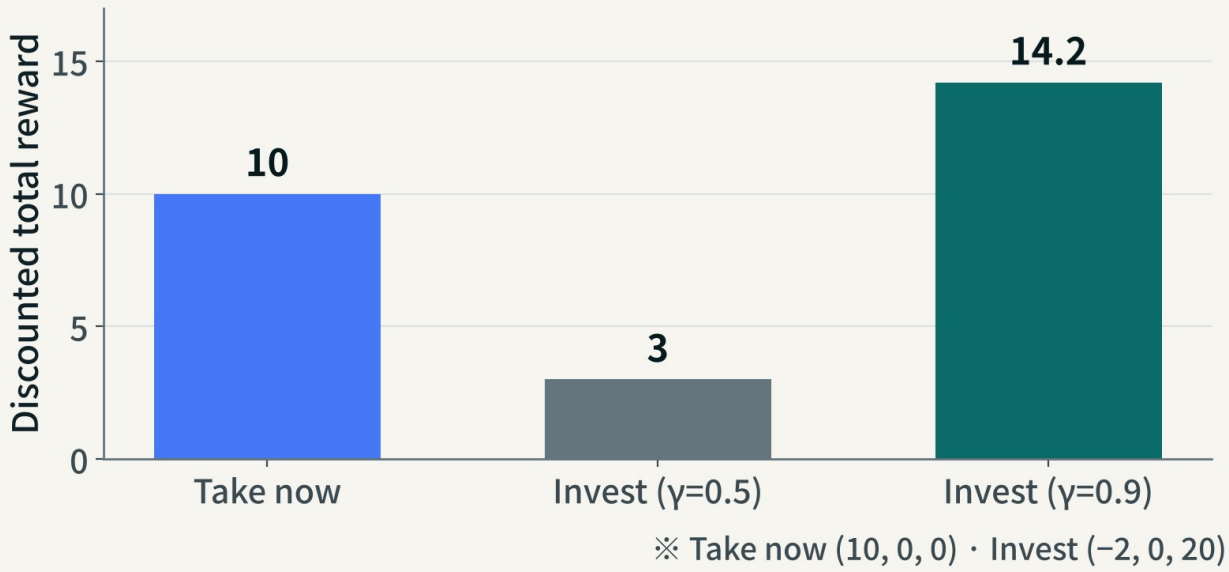
Choose Take now (10, 0, 0) or Invest (−2, 0, 20)

Discounted return

$$G_t = r_t + \underbrace{\gamma}_{\text{discount factor}} r_{t+1} + \gamma^2 r_{t+2} + \cdots$$

Reading

- At γ=0.5, Invest is worth 3, so Take now wins.
- At γ=0.9, it is 14.2 and the choice flips.



γ is how much future reward matters; not necessarily market rates

Mixing exploration and exploitation finds better options

ε-greedy: random action with prob. ε, else current best (2 actions, ε=0.2)

When A looks better (random pick includes A)

$$P(A) = 0.8 + \frac{0.2}{2} = 0.9, \qquad P(B) = 0.1$$

Exploitation

- One restaurant forever misses new ones.

Exploration

- Only new places wastes known good ones.

Exploration is not ‘picking bad actions’

Q-learning slowly updates a state-action score table

Old score 2, observed reward 1, next-state max 4, γ=0.9, α=0.5

Update

$$Q(s,a) \leftarrow Q(s,a) + \underbrace{\alpha}_{\text{learning rate}} \Big[\underbrace{r + \gamma \max_{a'} Q(s',a')}_{\text{new target}} - \underbrace{Q(s,a)}_{\text{old score}} \Big]$$

$$2 + 0.5\,(4.6 - 2) = 3.3, \qquad \text{new target} = 1 + 0.9 \times 4 = 4.6$$

α sets update speed, γ future weight, ε exploration rate: three separate knobs

Terminal states get 0 future reward; huge state spaces need approximation

Large orders are split between market impact and waiting cost

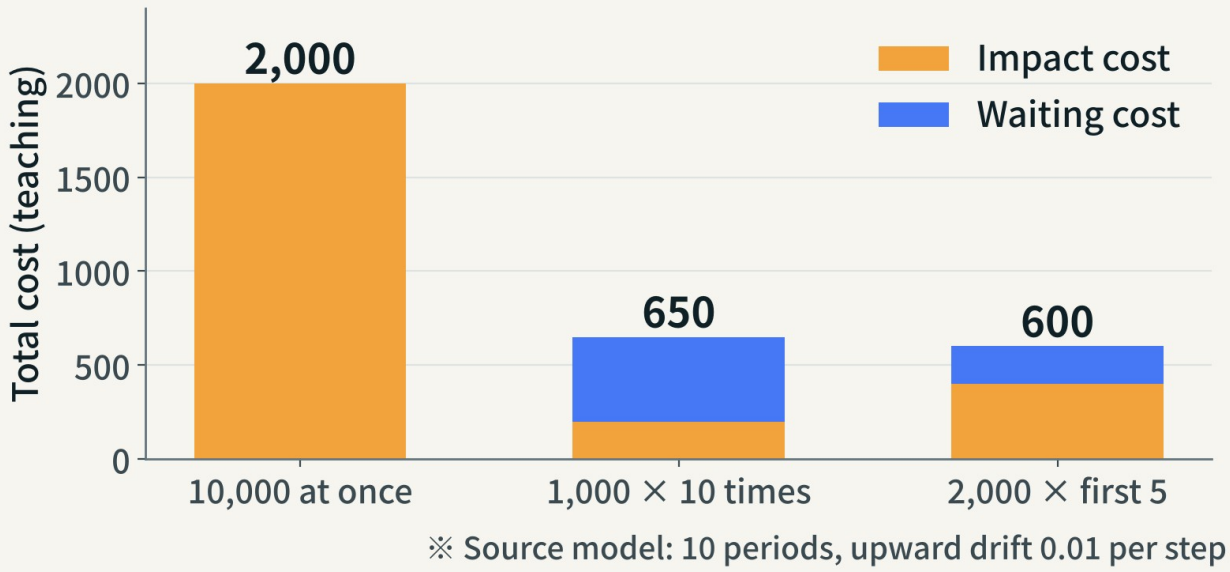
Source teaching cost model: 10,000 shares over 10 steps i=0,...,9

Total cost (q: size per step)

$$C = \sum_{i} \left(\underbrace{0.00002\, q_i^2}_{\text{impact cost}} + \underbrace{0.01\, i\, q_i}_{\text{waiting cost}} \right)$$

Reading

- TWAP splits evenly over time.
- RL can learn a policy adapting to time and shares left.



This table computes three fixed schedules; no proof RL learned an optimal one

Good rewards teach good actions

Agents learn to ignore what the reward leaves out

Profit-only

Excess leverage

- Ignoring risk, it raises leverage.

No costs

Overtrading

- Without costs, it trades needlessly often.

No fill penalty

Avoids orders

- With no penalty for unfilled shares, it avoids orders.

Compare candidates with simple baselines like TWAP under equal terms

High simulated returns alone do not mean live-ready

03

Lessons from cases

RL successes and its risks in markets

Winning at games does not mean winning in markets

AlphaGo (2016): won 4–1 against Lee Sedol 9-dan

Game

- The rules are fixed.
- The same setup can be simulated endlessly.

Market

- Rules and players keep changing.
- My orders move the price.

So

- First test how well the simulator fits reality.

A policy that shines in a wrong simulator fails live

A faulty execution algorithm loses big in minutes

Knight Capital (August 1, 2012): a faulty order system deployment

What happened

- Unintended mass orders ran for about 45 minutes.
- It lost about $440 million.

Reward design

- Without limits, bad actions repeat endlessly.

Operating rules

- Order limits, kill switch, pre-release tests.

Learned policies still need human-set safeguards

Classic and RL execution have different trade-offs

Almgren and Chriss (2000): classic model balancing market impact and price risk

CLASSICAL

Classic execution

- Clear assumptions, easy to explain.
- Weak when assumptions fail.

Clarity

REINFORCEMENT

RL execution

- Adapts to time, size, liquidity.
- Relies on simulator quality.

Adaptivity

Adopt RL only if it beats TWAP and the classic model on equal terms

04

Summary

Close the week with key Q&A

Judge a policy by its actions' outcomes

One question, one answer for this week

Check question	Answer
Accurate forecast = good policy?	Costs and the action's market effect also count.
High simulated profit: ready to go live?	Retest on unused periods with real fills, costs, risk limits.
How do α, γ and ε differ?	Update speed, future weight and exploration rate.
Is the execution table an RL result?	No. It computes three fixed schedules.

What the reward omits, the agent learns to ignore

PREDICTION IS NOT ACTION.

REWARD WHAT YOU WANT.

LIMIT WHAT CAN GO WRONG.

Design rewards and
safeguards together.